REQ5 Fleeing State Design Rationale

FIT2099 Assignment 2 Stateful Creatures

This rationale explains the Fleeing State design for the SporeBeast FleeingState. The selected design separates threat detection, safety assessment, and route planning from state-transition control. It gives the creature deterministic, explainable escape behaviour while satisfying the REQ5 requirement that state changes have an immediate effect. The conclusion is that a composed State-pattern design is more maintainable and testable than embedding all fleeing rules in SporeBeast.

1 Scope and Requirement Traceability

Zizhuo Wang is responsible for the fleeing part of the four-state SporeBeast. The implementation is centred on FleeingState and the helper classes in game.states.fleeing. It integrates with the common CreatureState interface, SporeBeast transition controller, StackQueueValleyWorld placement, and the existing SporeTrap item.

REQ5 expectation	Design response	Evidence in implementation
Deterministic transitions	FleeingState counts nearby Player or Hostile actors; it does not use Random.	ThreatScanner scans radius two and getNextState applies fixed conditions.
At least two outgoing states	No threats returns to CamouflagedState; exactly one adjacent threat returns to TerritorialState; otherwise the creature remains fleeing.	getNextState returns BaseStates.getCamouflagedState, BaseStates.getTerritorialState, or this.
Transition effect	Entering fleeing deploys a one-time spore barrier on valid neighbouring tiles.	onEnter creates non-duplicate SporeTrap items only on empty, enterable tiles.
New involved behaviour	The escape planner searches reachable locations, excludes occupied or unsafe positions, and ranks routes deterministically.	EscapeRoutePlanner uses breadth-first search, SafetyEvaluator, and stable tie-breakers.
Integration	The world places SporeBeast and SporeBeast delegates one turn at a time to its current CreatureState.	Application creates StackQueueValleyWorld; StackQueueValleyWorld places SporeBeast; SporeBeast invokes state methods.

2 Selected Design

2.1 Responsibilities

- FleeingState owns the fleeing turn policy, deterministic state exits, and the one-time entry effect.
- ThreatScanner identifies nearby Player and Hostile actors while excluding the SporeBeast itself.
- SafetyEvaluator rejects charged Chargeable ground and Fire, then measures Chebyshev distance from each threat.
- EscapeRoutePlanner performs breadth-first search and chooses a route by maximum safety, shortest path, y-coordinate, then x-coordinate.

- SporeBeast remains the state-machine controller: it asks the active state for one next state, calls onEnter only after a change, and then requests one ordinary turn action.

2.2 Deterministic Transition Policy

FleeingState uses a radius of two tiles to assess the local threat situation. If no threat remains nearby, continued escape is unnecessary and the creature moves to CamouflagedState. If exactly one threat is adjacent, the creature changes to TerritorialState because a close, single opponent can be confronted. In every other case, including two or more threats or a single threat still at range, it remains in FleeingState and continues to seek distance. These conditions are deterministic and depend on the changing map state rather than probability.

Observed condition	Next state	Reason
No Player or Hostile actor within radius two	CamouflagedState	The danger has passed, so concealment is safer than unnecessary movement.
Exactly one threat within radius one	TerritorialState	A lone adjacent threat is a controlled confrontation rather than an outnumbering event.
Two or more threats within radius two	FleeingState	The creature remains outnumbered and preserves its escape behaviour.
Exactly one threat at radius two	FleeingState	The creature maintains distance until the local situation changes.

2.3 Entry Effect

The state change is not only a label change. When SporeBeast enters FleeingState, onEnter examines every adjacent exit and places a SporeTrap only where the destination is empty, enterable, and does not already contain one. This creates a temporary tactical barrier around the escape point without overwriting actors, blocking terrain, or duplicating hazards. The effect runs once because SporeBeast calls onEnter only when the current-state reference changes.

3 Comparison of Alternative Designs

3.1 Design 1 Selected Composed State Design

Design 1 implements FleeingState as a CreatureState and composes it with ThreatScanner, SafetyEvaluator, and EscapeRoutePlanner. Each collaborator has one focused purpose. SporeBeast coordinates the transition but does not calculate threats or routes itself.

Pros	Cons
Strong separation of concerns: sensing, safety rules, and path selection can change independently.	More classes must be understood before a new team member can follow the full fleeing flow.
Deterministic tie-breakers make the selected escape route reproducible for demonstrations and tests.	BFS evaluates more locations than a simple adjacent-tile heuristic, although the game maps are small.
SafetyEvaluator can be reused by future creatures or states that need to avoid fire and charged terrain.	The state relies on the shared SporeTrap behaviour, so its trap semantics must remain compatible.
Unit-level testing is possible by supplying different scanner	The constructor dependency structure is slightly more

or route-planner collaborators.	verbose than direct calls inside one class.

3.2 Design 2 Rejected Centralised SporeBeast Logic

Design 2 would place threat scanning, hazard checks, route selection, transition conditions, and trap placement directly in SporeBeast.playTurn using a large conditional block. This is a plausible early implementation, but it is unsuitable for a four-state creature that will be extended by several team members.

Pros	Cons
Fewer source files and a short initial implementation path.	Violates Single Responsibility because SporeBeast would become actor, state machine, scanner, route planner, and hazard evaluator.
All state-related logic is visible in one method during a quick prototype.	High connascence of position, threat radius, hazard, and route-order rules makes changes risky.
No constructor collaborators are required.	Hard to test individual decisions; every test would need a complete SporeBeast and map scenario.
Can appear simpler for one state.	Becomes a long branching method as camouflage, foraging, territorial, and future states are added.

3.3 Decision

Design 1 was selected because it keeps the state transition protocol stable while allowing the fleeing algorithm to grow without modifying SporeBeast. It supports the group's division of work: Zizhuo Wang can improve escape logic without changing the code owned by members responsible for camouflage, foraging, or territorial behaviour.

4 Application of Design Principles

4.1 SOLID

Principle	Application in the fleeing design
Single Responsibility	ThreatScanner detects threats, SafetyEvaluator judges safety, EscapeRoutePlanner selects a route, and FleeingState controls state-specific policy.
Open Closed	New hazards can be added by extending SafetyEvaluator, and future state classes can implement CreatureState without rewriting SporeBeast.playTurn.
Liskov Substitution	FleeingState is usable wherever CreatureState is expected because it provides createTurnAction, getNextState, and onEnter according to the common contract.
Interface Segregation	CreatureState exposes only the state behaviour required by SporeBeast rather than forcing helper classes to implement unrelated methods.
Dependency Inversion	FleeingState depends on focused collaborators supplied through its constructor, allowing controlled replacement for tests and future strategy changes.

4.2 DRY KISS and Connascence

DRY is applied by centralising threat classification in ThreatScanner and safety checks in SafetyEvaluator. The same threat list is converted to locations once before route planning, rather than recalculating hazard logic at every call site. KISS is maintained by expressing the transition policy as three readable outcomes: safe, lone adjacent threat, or continued danger. Connascence is reduced because coordinates, distance measurement, and route ranking are encapsulated in one planner rather than copied into multiple state classes.

5 UML and Sequence Diagram Alignment

The updated UML class diagram includes Application and StackQueueValleyWorld to show how the stateful creature is created by the game entry point and placed in the world. It then shows SporeBeast holding a CreatureState, FleeingState realising that interface, and FleeingState using ThreatScanner and EscapeRoutePlanner. EscapeRoutePlanner uses SafetyEvaluator, while FleeingState creates SporeTrap items during onEnter.

The sequence diagram follows the same runtime order: World invokes SporeBeast.playTurn; SporeBeast asks FleeingState for the next state; an entry effect occurs only after a state change; then FleeingState scans threats, plans the safest route, and returns either a move action or DoNothingAction. This alignment makes the design documents traceable to the code.

6 Verification Strategy

Scenario	Expected result	Purpose
No threats within two tiles	FleeingState returns CamouflagedState; no escape move is generated.	Verifies the safe exit path.
One adjacent Farmer	FleeingState returns TerritorialState.	Verifies the lone close-threat path.
Two nearby threats	FleeingState remains active, deploys its barrier on entry, and chooses the safest reachable route.	Verifies the main fleeing condition and entry effect.
Fire or charged puddle on a possible route	SafetyEvaluator rejects the hazardous tile even if it is farther from a threat.	Verifies environmental safety has priority.
Map edge or no safe destination	EscapeRoutePlanner returns no exit and FleeingState returns DoNothingAction without an exception.	Verifies boundary and no-route resilience.
Repeated entry to fleeing	A tile already containing SporeTrap is not given a duplicate trap.	Verifies idempotent entry-effect behaviour.

7 Conclusion

The revised FleeingState meets Zizhuo Wang's responsibility by combining nearby-threat sensing, safe-location evaluation, and deterministic route selection. It now has two explicit outgoing transitions, a meaningful staying condition, and a one-time spore-barrier effect when entering the state. The design is modular enough to support future hazards, maps, or creatures while remaining consistent with the group's State-pattern framework.